

WarehouseFlow — Changelog

Week-by-week progression of the WarehouseFlow distributed order routing engine.

[Week 1] — Infrastructure & End-to-End Skeleton

Added

- **Ingestion service** — Go HTTP service (gorilla/mux), UUID assignment, Kafka publisher with hash partitioning
- **Routing service** — Go Kafka consumer, routing engine (Ping → inventory → picker → atomic decrement → audit log)
- **Store package** — Redis warehouse registry with Lua atomic decrement + picker queue, Postgres audit log
- **DLQ** — SQS dead-letter queue integration
- **Docker Compose** — Redpanda + 3x Redis + Postgres + Prometheus + Grafana
- **Terraform** — Full AWS stack (VPC, ECS Fargate + ALB, MSK, ElastiCache x3, RDS, SQS, ECR)
- **Unit tests** — miniredis-based, no Docker required
- **Smoke test** — End-to-end order flow verification

Exit Criterion

Single order flows end-to-end: Locust → Kafka → routing → Postgres

[Week 2] — Observability & Experiment 1

Added

- Grafana dashboard JSON with 8 panels (throughput, P50/P95/P99, per-warehouse, CB state, bulkhead, etc.)
- Locust load profile for Experiment 1 (steady 5,000 orders/min)
- Runner script + chart generator for Experiment 1
- CSV results + PNG charts for horizontal scaling + saturation sweep
- [docs/week2_analysis.md](#)

Key Findings

- 4 ECS tasks = 5,000 orders/min at P99 = 32.8ms (below 200ms SLA)
 - ElastiCache becomes bottleneck at 8 tasks
 - Saturation at ~5,500 orders/min
-

[Week 3] — Contention & Resilience

Added

- **Optimistic routing engine** — CAS retry with jitter
- **Pessimistic routing engine** — Redis SETNX distributed lock
- **Strategy pipeline** — shared `routeWithStrategy` for DRY
- **Failure injection** — `inject_failure.sh` + `restore_warehouse.sh`
- Runner scripts + Locust files for Experiments 2 and 3
- `docs/week3_analysis.md`

Key Findings

- **Experiment 2:** Failover 3.2s, recovery 6.1s, zero orders lost, all 24 DLQ orders reprocessed
- **Experiment 3:** Both strategies achieve zero oversells — Lua atomic decrement is the real guarantee
- Optimistic 2.3x faster (187 vs 80 orders/sec), 41.5% retry rate

[Week 4] — Resilience Patterns + 4 New Experiments + Interactive Dashboard

Added — Resilience Pattern Suite

- **Circuit Breaker** — CLOSED/OPEN/HALF-OPEN state machine + tests
- **Bulkhead** — Per-warehouse counting semaphore + tests
- **Retry** — Exponential backoff with $\pm 50\%$ jitter + tests
- **Fail-Fast** — Bounded context timeouts (Redis/Postgres/Kafka/SQS)
- **Degradation** — Local buffer fallback + tests
- Prometheus metrics for all patterns

Added — 4 New Experiments

- **Experiment 4** — Network partition & eventual consistency (CAP)
- **Experiment 5** — Kafka partition count vs consumer parallelism
- **Experiment 6** — Noisy neighbor tail latency
- **Experiment 7** — Cold start after auto-scale

Added — Interactive Dashboard

- `dashboard/index.html` · `styles.css` · `app.js`
- Industrial dark theme (signal colors only)
- Live SVG warehouse map with routing particles
- 7-experiment autopilot + failure injection buttons
- Self-contained simulator (no backend required)

Key Findings

- **Exp 4:** Zero orders lost during 120s partition, 20s reconciliation, AP chosen
- **Exp 5:** Throughput capped by partition count, 12 partitions → 4x scaling
- **Exp 6:** Shared ElastiCache causes 4x P99 spike on unrelated SKUs
- **Exp 7:** Cold tasks peak at 178ms P99 (4.2x warm baseline)
- **Resilience:** Circuit breaker converts 2,100ms slow-dep P99 to 42ms

Exit Criterion

All 7 experiments complete · full resilience suite integrated · interactive dashboard deployed · final report written · cloud destroyed
